



NuMagSANS Tutorial 3

NuMagSANS Tutorials - Michael P. Adams

Version: October 2, 2026

Running the Tutorials

The executable tutorial scripts support two installation modes.

Shared setup: local workstation or HPC shell

`python -m venv .venv`

`source .venv/bin/activate`

Core install: data generation and NuMagSANS runs

`pip install -e .`

Optional local plotting support

`pip install -e ".[tutorials]"`

- Use the core install for batch/HPC runs.
- Missing PyVista only disables diagnostic plots.
- Use the tutorials extra for local vector-field inspection.

Generate one spherical object and compare several local magnetization models.

- The script writes one shared local sphere.
- Each parameter set writes one file `m_i.csv` in `MagData/Object_1`.
- Static PNG diagnostics are written for quick inspection and slides.

The spherical library combines a geometry, a radial profile, and a field ansatz.

field_type	coordinate input	profile	main parameters
vortex	ρ, φ	$\alpha(\xi)$	xi_type, profile_type, kappa, turns, normalize
hedgehog	r, θ, φ	$\alpha(\xi)$	xi_type, profile_type, kappa, turns, normalize
artichoke	r, θ, φ	$\alpha(\xi)$	xi_type, profile_type, kappa, turns, normalize
poloidal_vortex	r, θ, φ	$\alpha(\xi)$	xi_type, profile_type, kappa, turns, normalize
skyrmion	ρ, φ	built-in	N, m, gamma, xi_type
transversal_helix	z	none	k, chirality, phase
longitudinal_helix	z	none	k, chirality, phase, lambda
linearized_vortex	x, y	none	m_0, m_1

- The profile-based fields share the same $\alpha(\xi)$ interface.
- `normalize=True` normalizes the final vector field; set `normalize=False` to keep raw amplitudes.

Profile-Based Spherical Fields

For the profile-based fields, the same radial tilt is inserted into different bases.

$$\xi = \begin{cases} \rho/R, & \text{cylindrical_xi} \\ r/R, & \text{spherical_xi} \end{cases} \quad \alpha = \alpha(\xi)$$

$$\text{vortex} : \sin \alpha \mathbf{e}_\varphi + \cos \alpha \mathbf{e}_z$$

$$\text{hedgehog} : \sin \alpha \cos \theta \mathbf{e}_r + \cos \alpha \mathbf{e}_z$$

$$\text{artichoke} : -\sin \alpha \mathbf{e}_\theta + \cos \alpha \mathbf{e}_z$$

$$\text{poloidal_vortex} : \sin \alpha \mathbf{e}_\theta + \cos \alpha \mathbf{e}_z$$

linear

parabolic

trigonometric

hyperbolic

argument_hyperbolic

lorentzian

gaussian

arctan

- turns multiplies α before the basis components are assembled.
- `normalize=True` normalizes the result after the field is evaluated.

Some spherical-library fields define the vector components directly.

skyrmion:	$(\sin(\pi N\xi) \cos(m\varphi + \gamma), \sin(\pi N\xi) \sin(m\varphi + \gamma), \cos(\pi N\xi))$	N, m, γ
transversal_helix:	$\mathbf{e}_x \cos(kz + \varphi_0) + \chi \mathbf{e}_y \sin(kz + \varphi_0)$	k, χ, φ_0
longitudinal_helix:	$\mathbf{m}'_{\text{helix}} + \lambda \mathbf{e}_z, \quad \mathbf{m} = \mathbf{m}'/ \mathbf{m}' $	$k, \chi, \varphi_0, \lambda$
linearized_vortex:	$(-m_1 y, m_1 x, m_0)$	m_0, m_1

- Useful when the texture is not primarily controlled by a radial tilt profile.
- The final normalization can be disabled with `normalize=False`.

Operator-Kernel Approach

The operator-kernel library builds each component from a differential operator applied to a scalar kernel.

$$\mathbf{m}'(\mathbf{r}) = \begin{pmatrix} P_x(\partial_x, \partial_y, \partial_z)K(\mathbf{r}) \\ P_y(\partial_x, \partial_y, \partial_z)K(\mathbf{r}) \\ P_z(\partial_x, \partial_y, \partial_z)K(\mathbf{r}) \end{pmatrix}, \quad \mathbf{m} = \mathbf{m}'/|\mathbf{m}'|$$

- P_x, P_y, P_z are polynomials in the derivative symbols $\partial_x, \partial_y, \partial_z$.
- `component_specs` stores these three polynomials as strings in `dx, dy, dz`.
- In Fourier space, derivatives correspond to multipliers iq_x, iq_y, iq_z .
- `normalize=True` normalizes $\mathbf{m}'$ after all components are evaluated.

Operator Specifications

Example component specifications for a vortex-like operator-kernel field.

$$\text{"-a * dy"} \Rightarrow P_x = -a \partial_y \Rightarrow m'_x = -a \partial_y K$$

$$\text{"a * dx"} \Rightarrow P_y = a \partial_x \Rightarrow m'_y = a \partial_x K$$

$$\text{"c"} \Rightarrow P_z = c \Rightarrow m'_z = c K$$

- Higher derivatives are possible, e.g. dx^2 , dy^2 , or mixed operator polynomials.
- Coefficients such as a, c, d are supplied through `operator_parameters`.
- The raw field can be kept with `normalize=False`.

The kernel controls the localization; the component operators control the vector texture.

$$K_{\text{Gaussian}}(\mathbf{r}) \propto \exp\left[-\frac{1}{2}\mathbf{r}^T \Sigma^{-1} \mathbf{r}\right]$$

$$K_{\text{sech}}(\mathbf{r}) = \prod_{i=x,y,z} \frac{a_i/\pi}{\cosh(a_i r_i)}$$

- `kernel_type="gaussian"` uses `sigma` or a full covariance matrix `Sigma`.
- `kernel_type="sech"` uses one width parameter per coordinate direction.
- `coordinate_scale` rescales object coordinates before evaluating the kernel.

Run the Survey

Run the executable tutorial from the NuMagSANS repository root.

```
python tutorials/tutorial_3/vectorfield_survey.py
```

The generated data are written to `tutorials/tutorial_3/VectorFieldSurvey`.

The same spherical object is reused for all vector-field parameter sets.

```
write_spherical_replication_vectorfield_sweep(  
    output_dir=OUTPUT_DIR,  
    R=20.0,  
    a=1.0,  
    atomtype="Fe",  
    n_replications=256,  
    field_parameter_cases=FIELD_PARAMETER_CASES,  
    n_rotdata=1,  
    beta_min=0.0,  
    beta_max=np.pi,  
    rotation_seed=1,  
)
```

Each entry in FIELD_PARAMETER_CASES writes one magnetization file `m_i.csv`.

The first four fields use a hyperbolic tilt profile.

$$\alpha(\xi) = \arccos\left(\sqrt{1 - \tanh^2(\kappa\xi)}\right), \quad \kappa = 6, \quad \text{turns} = 1.$$

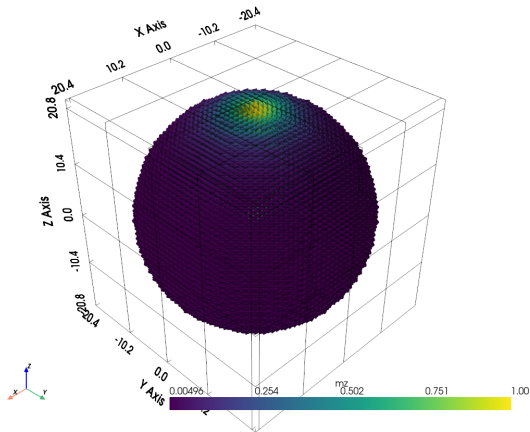
- `cylindrical_xi` uses $\xi = \rho/R$.
- `spherical_xi` uses $\xi = r/R$.

Hyperbolic Vortex

```
{"field_type": "vortex",  
  "profile_type": "hyperbolic",  
  "xi_type": "cylindrical_xi",  
  "kappa": 6.0}
```

$$\mathbf{m} = \sin \alpha(\rho/R) \mathbf{e}_\varphi + \cos \alpha(\rho/R) \mathbf{e}_z$$

- cylindrical radial profile
- azimuthal in-plane winding
- finite m_z core

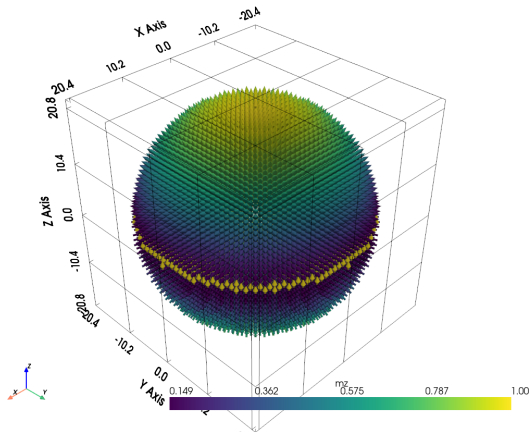


Hyperbolic Hedgehog

```
{"field_type": "hedgehog",  
  "profile_type": "hyperbolic",  
  "xi_type": "spherical_xi",  
  "kappa": 6.0}
```

$$\mathbf{m} = \sin \alpha(r/R) \cos \theta \mathbf{e}_r + \cos \alpha(r/R) \mathbf{e}_z$$

- spherical radial profile
- radial component weighted by $\cos \theta$
- axial background contribution

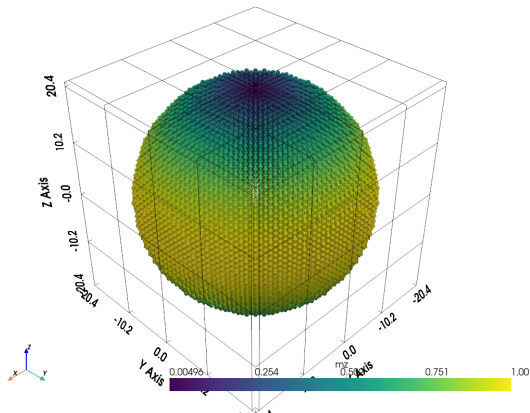


Hyperbolic Artichoke

```
{"field_type": "artichoke",  
  "profile_type": "hyperbolic",  
  "xi_type": "spherical_xi",  
  "kappa": 6.0}
```

$$\mathbf{m} = -\sin \alpha(r/R) \mathbf{e}_\theta + \cos \alpha(r/R) \mathbf{e}_z$$

- spherical radial profile
- negative polar component
- compact onion-like texture

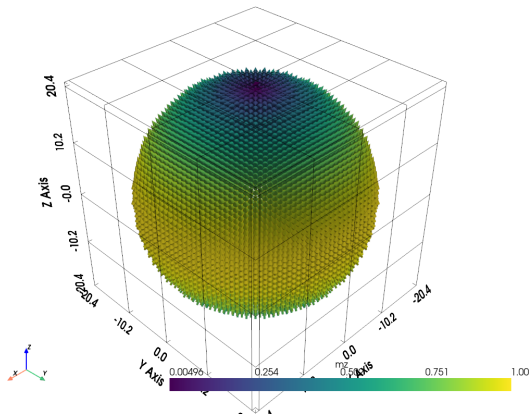


Hyperbolic Poloidal Vortex

```
{"field_type": "poloidal_vortex",  
  "profile_type": "hyperbolic",  
  "xi_type": "spherical_xi",  
  "kappa": 6.0}
```

$$\mathbf{m} = \sin \alpha(r/R) \mathbf{e}_\theta + \cos \alpha(r/R) \mathbf{e}_z$$

- spherical radial profile
- positive polar circulation
- poloidal winding around the sphere

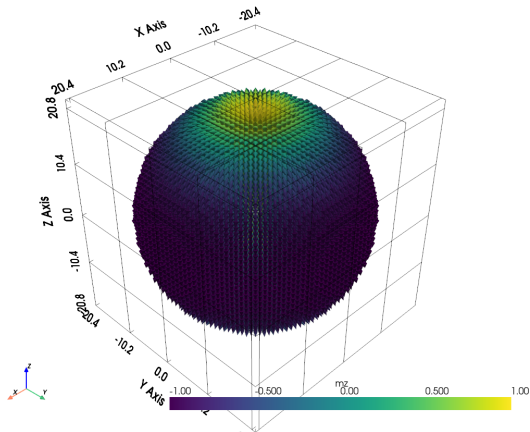


Skyrmion

```
{"field_type": "skyrmion",  
  "xi_type": "cylindrical_xi",  
  "N": 1, "m": 1,  
  "gamma": 0.0}
```

$$\mathbf{m} = \begin{pmatrix} \sin(\pi N \xi) \cos(m\varphi + \gamma) \\ \sin(\pi N \xi) \sin(m\varphi + \gamma) \\ \cos(\pi N \xi) \end{pmatrix}$$

- $\xi = \rho/R$
- $N = 1, m = 1, \gamma = 0$
- radial reversal in m_z

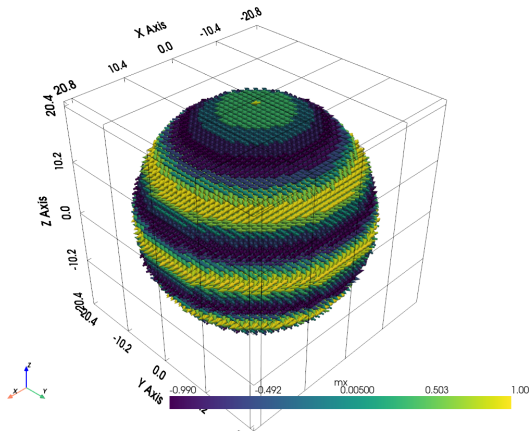


Transversal Helix

```
{"field_type": "transversal_helix",  
  "k": 0.6,  
  "chirality": 1.0,  
  "phase": 0.0}
```

$$\mathbf{m}(z) = \mathbf{e}_x \cos(kz + \varphi_0) + \chi \mathbf{e}_y \sin(kz + \varphi_0)$$

- $k = 0.6$
- $\chi = 1, \varphi_0 = 0$
- transverse rotation along z



Longitudinal Helix

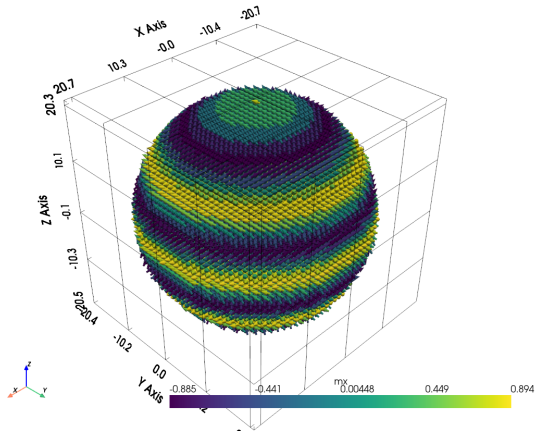
```
{"field_type": "longitudinal_helix",  
  "k": 0.6,  
  "chirality": 1.0,  
  "lambda": 0.5,  
  "phase": 0.0}
```

$$\mathbf{m}'(z) = \mathbf{e}_x \cos(kz + \varphi_0)$$

$$+ \chi \mathbf{e}_y \sin(kz + \varphi_0) + \lambda \mathbf{e}_z,$$

$$\mathbf{m} = \mathbf{m}' / |\mathbf{m}'|$$

- $\lambda = 0.5$
- transverse helix plus constant z-component
- normalized after adding the longitudinal term



Linearized Vortex

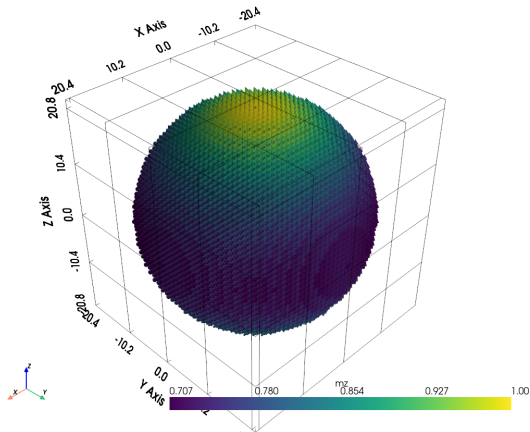
```
{"field_type": "linearized_vortex",
```

```
  "m_0": 1.0,
```

```
  "m_1": 0.05}
```

$$\mathbf{m}' = \begin{pmatrix} -m_1 y \\ m_1 x \\ m_0 \end{pmatrix}, \quad \mathbf{m} = \mathbf{m}' / |\mathbf{m}'|$$

- $m_0 = 1$, $m_1 = 0.05$
- linear in-plane circulation
- normalized after evaluation

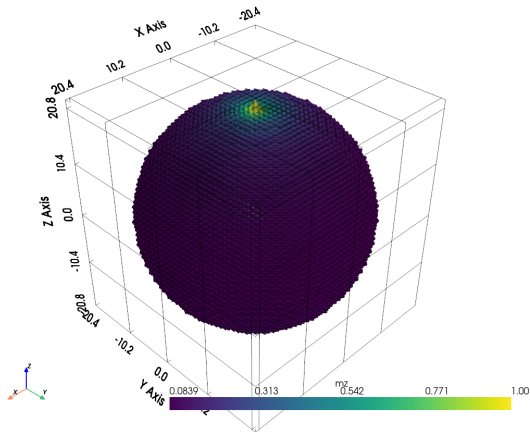


Operator-Kernel Gaussian Vortex

```
{"library": "operator_kernel",  
  "kernel_type": "gaussian",  
  "component_specs": ["-a * dy", "a *  
dx", "c"],  
  "operator_parameters": {"a": 0.8, "c":  
0.55}}
```

$$\mathbf{m}' = \begin{pmatrix} -a \partial_y K \\ a \partial_x K \\ c K \end{pmatrix}, \quad K = K_{\text{Gaussian}}$$

- $\sigma = (0.35, 0.35, 0.35)$
- $a = 0.8, c = 0.55$
- differential-operator texture

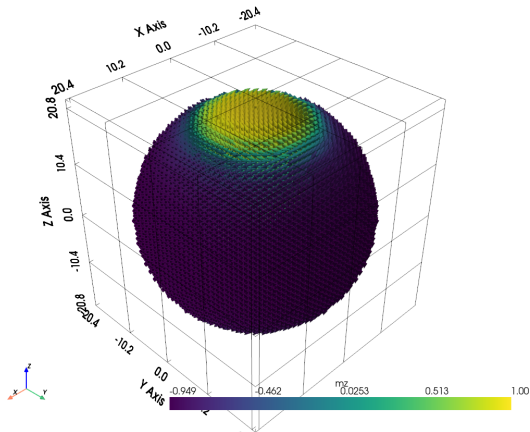


Operator-Kernel Gaussian Skyrmion-Like

```
{"kernel_type": "gaussian",  
  "component_specs": [  
    "-a * dy", "a * dx",  
    "c + d * (dx^2 + dy^2)"  
  ]}
```

$$\mathbf{m}' = \begin{pmatrix} -a \partial_y K \\ a \partial_x K \\ (c + d(\partial_x^2 + \partial_y^2)) K \end{pmatrix}$$

- Gaussian kernel
- $a = 0.7$, $c = -1.6$, $d = -0.5$
- skyrmion-like axial reversal



Operator-Kernel Sech Vortex

```
{"library": "operator_kernel",  
  "kernel_type": "sech",  
  "a": [2.0, 2.0, 2.4],  
  "component_specs": ["-a * dy", "a *  
dx", "c"]}
```

$$\mathbf{m}' = \begin{pmatrix} -a \partial_y K \\ a \partial_x K \\ c K \end{pmatrix}, \quad K = K_{\text{sech}}$$

- $a_{\text{kernel}} = (2.0, 2.0, 2.4)$
- $a = 0.8, c = 0.55$
- broader operator-kernel vortex

